

Classes, Models, and the Training Loop

Advanced Machine Learning

Sam Ogden

DRAFT

This slide deck is still under active development. Expect changes before it is finalized.

Learning objectives

- Explain what a Python class bundles together, and read `__init__`, `self`, `super()`, and `__call__` in code
- State the model interface used all semester: an `nn.Module` subclass with `__init__` (create parameters) and `forward` (inputs to predictions)
- Build a linear model, combine several into a multi-output model, and layer them into an MLP, all with the same interface
- Say which object does which job in a training loop, and write the loop from memory
- Recognize d2l's `training_step` / `loss` / `configure_optimizers` as optional packaging on top of that interface

Object-Oriented Python

Bundling data with behavior

Why classes?

Answering “are all models going to be classes?”

Why a model wants to be a class

- A model is really **two things**:
 - Numbers it remembers (the parameters W, b)
 - A recipe that uses them (the forward pass)
- A class bundles both of these things
 - A function alone forgets its numbers between calls

Anatomy of a class

```
1 class Line:
2     def __init__(self, w, b):
3         self.w = w
4         self.b = b
5
6     def predict(self, x):
7         return self.w * x + self.b
8
9 line = Line(2, 1)
10 print(line.predict(3))
```

7

- `Line(2, 1)` builds an object; `line.predict(3)` calls a method on it
- The object remembers `w` and `b` between calls

Breakdown: `__init__`

```
1 def __init__(self, w, b):  
2     self.w = w  
3     self.b = b
```

- Runs automatically when we write `Line(2, 1)`: it is the **constructor**
- It sets up the object's **attributes**, the numbers the object remembers
- No field *declarations*: attributes exist the moment we assign to `self.something`

Breakdown: `self`

Java

```
1 public double predict(double x) {  
2     return this.w * x + this.b;  
3 }
```

- `this` is implicit

Python

```
1 def predict(self, x):  
2     return self.w * x + self.b
```

- `self` is written out as the first parameter
- Python passes it for us:
 `line.predict(3)` becomes
 `Line.predict(line, 3)`

Java vs. Python: quick translation

	Java	Python
Constructor	same name as the class	<code>__init__</code>
Current object	<code>this</code> (implicit)	<code>self</code> (explicit first parameter)
Creating an object	<code>new Line(2, 1)</code>	<code>Line(2, 1)</code>
Fields	declared up front	created by assigning to <code>self.x</code>
Private	<code>private</code> keyword	convention only: <code>_name</code>

Dunder methods

- Names with *double underscores* on both sides: `__init__`, `__call__`, `__len__`, ...
- We never call them by name. **Python calls them for us** when we use certain syntax:

We write	Python runs
<code>Line(2, 1)</code>	<code>Line.__init__(...)</code>
<code>len(x)</code>	<code>x.__len__()</code>
<code>line(3)</code>	<code>line.__call__(3)</code>

`__call__`: objects that act like functions

```
1 class Line:
2     def __init__(self, w, b):
3         self.w = w
4         self.b = b
5
6     def predict(self, x):
7         return self.w * x + self.b
8
9     def __call__(self, x):
10        return self.predict(x)
11
12 line = Line(2, 1)
13 print(line(3))
```

7

- `line(3)` looks like a function call, but `line` is an object holding w and b
- This is exactly why we will write `model(X)` for a model

Inheritance and `super()`

```
1 class ReluLine(Line):
2     def __init__(self, w, b):
3         super().__init__(w, b)
4
5     def __call__(self, x):
6         return max(0, super().__call__(x))
7
8 print(Line(2, -10)(3), ReluLine(2, -10)(3))
```

-4 0

- `ReluLine(Line)` is a `Line`, and gets its methods for free
- `super()` reaches the parent's version: here we reuse `Line`'s setup and its `__call__`, then clip at zero
- `super().__init__(...)` is essential!

⚠ Important

Forgetting `super().__init__(...)` means the parent never sets up its part. With `nn.Module` this is a real bug, as we'll see.

Reference: Python classes

```
1 class Line:
2     def __init__(self, w, b):          # constructor: remember the numbers
3         self.w = w
4         self.b = b
5
6     def predict(self, x):              # a method: uses the remembered numbers
7         return self.w * x + self.b
8
9     def __call__(self, x):             # dunder: makes line(3) work
10        return self.predict(x)
11
12
13 class ReluLine(Line):                 # inheritance: a ReluLine IS a Line
14     def __init__(self, w, b):
15         super().__init__(w, b)        # let the parent set up its part
16
```

The model interface

What we expect of every model

What do we need from a model?

- Given an input X , produce predictions $\hat{y}$
- Hold **parameters** that training can change
- Be **easy to use**: move to a GPU, save it, switch it between training and evaluation
- Work with **any** optimizer and any loss without special cases

The Big Idea

A model is an object that owns its parameters and knows how to turn inputs into predictions. Everything else comes for free.

The minimal model

```
1 import torch
2 from torch import nn
3
4 class Model(nn.Module):
5     def __init__(self):
6         super().__init__()          # gives us everything nn.Module can do
7         # create parameters / sub-models here
8
9     def forward(self, X):
10         """Given X, return y_hat."""
11         ...
```

- Two methods: `__init__` builds, `forward` computes
- `nn.Module` is the base class: we don't have to write anything else

What `nn.Module` gives us for free

- `model(X)`: run the forward pass (through `__call__`)
- `model.parameters()`: every trainable number, ready to hand to an optimizer
- `model.to(device)`: move everything to the GPU in one call
- `model.state_dict()`: save and load the parameters
- `model.train()` / `model.eval()`: switch modes

Note

None of this is in our code. We inherit it by writing `nn.Module` in the class line and calling `super().__init__()`.

`model(X)` is `__call__` again

Our Line

```
1 def __call__(self, x):  
2     return self.predict(x)
```

`nn.Module` (simplified)

```
1 def __call__(self, *args):  
2     # bookkeeping and hooks  
3     return self.forward(*args)
```

- So `model(X)` runs `model.forward(X)` plus PyTorch's bookkeeping
- Always write `model(X)`, never `model.forward(X)`: calling `forward` directly skips the bookkeeping

Three rules for a model class

1. Call `super().__init__()` first, before assigning any parameters or sub-models
 2. Register what training must change: wrap raw tensors in `nn.Parameter`, and assign sub-models as attributes
 3. Call `model(X)`, not `model.forward(X)`
- Rule 2 is the one that bites. Next slide.

Gotcha: the invisible list

```
1 class Broken(nn.Module):
2     def __init__(self):
3         super().__init__()
4         self.layers = [nn.Linear(3, 3), nn.Linear(3, 1)]      # plain list
5
6 class Fixed(nn.Module):
7     def __init__(self):
8         super().__init__()
9         self.layers = nn.ModuleList([nn.Linear(3, 3), nn.Linear(3, 1)])
10
11 print(len(list(Broken().parameters())), len(list(Fixed().parameters())))
```

0 4

- A plain Python list is invisible to `nn.Module`: the optimizer would get **zero** parameters and nothing would ever train
- `nn.ModuleList` is a list that `nn.Module` can see into

Reference: the model interface

```
1 import torch
2 from torch import nn
3
4 class Model(nn.Module):
5     def __init__(self):
6         super().__init__()          # rule 1: always first
7         # rule 2: parameters as nn.Parameter, sub-models as attributes
8         #         (use nn.ModuleList for a list of sub-models)
9
10    def forward(self, X):
11        """Given X, return y_hat."""
12        ...
13
14 model = Model()
15 y_hat = model(X)                    # rule 3: call the model, not .forward
16 model.parameters()                 # everything trainable, found for us
```

Building models

From one line to an MLP

The Big Idea

A model can contain other models, and every one of them has the same interface, so we can stack them like blocks.

Model 1: linear regression, from scratch

```
1 class LinearRegression(nn.Module):
2     def __init__(self, num_features):
3         super().__init__()
4         self.w = nn.Parameter(torch.randn(num_features))
5         self.b = nn.Parameter(torch.zeros(1))
6
7     def forward(self, X):
8         return X @ self.w + self.b
```

- The same $\hat{y} = \mathbf{w} \cdot \mathbf{x} + b$ from earlier, now with a home for w and b
- `nn.Parameter` marks a tensor as “training may change this”

Worked example: the high-temperature model

```
1 high = LinearRegression(3)
2 with torch.no_grad(): # overwrite the random start with our earlier numbers
3     high.w.copy_(torch.tensor([0.8, 0.1, -0.05]))
4     high.b.fill_(10.0)
5
6 x = torch.tensor([[75.0, 55.0, 40.0]])
7 print(high(x))
```

tensor([73.5000], grad_fn=<AddBackward0>)

- Same $w = [0.8, 0.1, -0.05]$, $b = 10$ and $x = [75, 55, 40]$: the prediction is 73.5, exactly the earlier number
- `model(x)` is the whole interface

PyTorch already has this one

```
1 layer = nn.Linear(3, 1)
2 print([(name, p.shape) for name, p in layer.named_parameters()])
```

`[('weight', torch.Size([1, 3])), ('bias', torch.Size([1]))]`

- `nn.Linear(3, 1)` is our `LinearRegression` (weights stored as a 1×3 matrix)
- `nn.Linear(3, 2)` would be a **two-output** model in one object: a 2×3 W and two biases
- It is an `nn.Module` too, so it has the same interface as ours

Model 2: several models, one model

```
1 class ParallelLinear(nn.Module):
2     def __init__(self, num_features, num_outputs):
3         super().__init__()
4         self.models = nn.ModuleList(
5             [LinearRegression(num_features) for _ in range(num_outputs)]
6         )
7
8     def forward(self, X):
9         return torch.stack([m(X) for m in self.models], dim=-1)
```

- A model that **contains** models: one `LinearRegression` per output
- This is the earlier “one model per target” idea, now packaged as a single object

Worked example: high and low

```
1 both = ParallelLinear(3, 2)
2 with torch.no_grad():
3     both.models[0].w.copy_(torch.tensor([0.8, 0.1, -0.05]))
4     both.models[0].b.fill_(10.0)
5     both.models[1].w.copy_(torch.tensor([0.3, 0.6, -0.02]))
6     both.models[1].b.fill_(5.0)
7
8 print(both(x))
```

tensor([[73.5000, 59.7000]], grad_fn=<StackBackward0>)

- [73.5, 59.7]: the same two numbers as before, from **one** call
- Every parameter is found automatically: `len(list(both.parameters()))` is 4

Models are blocks

Model 3: layer them into an MLP

```
1 import torch.nn.functional as F
2
3 class MLP(nn.Module):
4     def __init__(self, num_features, num_hidden, num_outputs):
5         super().__init__()
6         self.hidden = nn.Linear(num_features, num_hidden)
7         self.out = nn.Linear(num_hidden, num_outputs)
8
9     def forward(self, X):
10         return self.out(F.relu(self.hidden(X)))
11
12 mlp = MLP(3, 4, 2)
13 print(mlp(x).shape, sum(p.numel() for p in mlp.parameters()))
```

torch.Size([1, 2]) 26

- `__init__` creates the pieces, `forward` wires them together
- Called the same way: `mlp(x)`. 26 parameters: $3 \cdot 4 + 4$ in the hidden layer, $4 \cdot 2 + 2$ in the output

The same thing with `nn.Sequential`

```
1 mlp2 = nn.Sequential(nn.Linear(3, 4), nn.ReLU(), nn.Linear(4, 2))
```

```
2 print(mlp2(x).shape)
```

```
torch.Size([1, 2])
```

- `nn.Sequential` is itself a model whose `forward` runs its pieces in order
- Fine for a straight line of layers. Once the wiring branches or loops, we write our own `forward`

Reference: building models

```
1 class LinearRegression(nn.Module):                # 1. a single linear model
2     def __init__(self, num_features):
3         super().__init__()
4         self.w = nn.Parameter(torch.randn(num_features))
5         self.b = nn.Parameter(torch.zeros(1))
6
7     def forward(self, X):
8         return X @ self.w + self.b
9
10
11 class ParallelLinear(nn.Module):                  # 2. a model of models
12     def __init__(self, num_features, num_outputs):
13         super().__init__()
14         self.models = nn.ModuleList(
15             [LinearRegression(num_features) for _ in range(num_outputs)])
16
```

The training loop

Who does what

From predicting to learning

- A model with random parameters makes bad predictions
- We already know **how** to fix that: score the predictions with a loss, compute the gradient, take a step
- The question now is **which object** does each of those jobs

The Big Idea

Training is a loop where separate objects each do one job, and the loop itself only calls them in the right order.

Who does what

Job	Who does it	In code
Turn inputs into predictions	the model	<code>model(X)</code>
Score the predictions	the loss function	<code>loss_fn(y_hat, y)</code>
Compute gradients	autograd	<code>loss.backward()</code>
Update the parameters	the optimizer	<code>optimizer.step()</code>
Hand out mini-batches	the data loader	<code>for X, y in loader:</code>

- The model knows nothing about the optimizer, and the optimizer knows nothing about the model's structure. It only sees `model.parameters()`

The five lines

```
1 for X, y in loader:           # one mini-batch
2     optimizer.zero_grad()      # 1. clear old gradients
3     y_hat = model(X)          # 2. forward pass
4     loss = loss_fn(y_hat, y)   # 3. score it
5     loss.backward()            # 4. gradients, via autograd
6     optimizer.step()          # 5. update the parameters
```

- This is **the** training loop. Everything else in the course is a variation on these five lines

Breakdown: why `zero_grad()`?

- `loss.backward()` adds to each parameter's `.grad`, it does not overwrite it
- Without `zero_grad()`, gradients from the previous batch pile up and our step points the wrong way
- So: clear, then compute, then step. Every batch, in that order

Breakdown: `train()`, `eval()`, and `no_grad`

- **Training:** call `model.train()`. The loop above runs
- **Evaluating:** call `model.eval()` and wrap the pass in `torch.no_grad()`. We only want predictions, not gradients
- Some layers (dropout, batch norm) **behave differently** in the two modes. `.train()` / `.eval()` is how we tell them, and it is one of the free `nn.Module` methods

Same loop, new clothes

The math we already know

- Pick a mini-batch
- Compute predictions $\hat{y} = Wx + b$
- Compute the gradient of the loss
- $W \leftarrow W - \eta \nabla_W L$

- Nothing new was introduced: it's mini-batch SGD written as code

The code

- `for X, y in loader:`
- `y_hat = model(X)`
- `loss.backward()`
- `optimizer.step()` (with `lr =  $\eta$`)

Worked example: train the high/low model

```
1 torch.manual_seed(0)
2 W_true = torch.tensor([[0.8, 0.1, -0.05], [0.3, 0.6, -0.02]])
3 b_true = torch.tensor([10.0, 5.0])
4 X_all = torch.randn(256, 3)
5 Y_all = X_all @ W_true.T + b_true + 0.1 * torch.randn(256, 2)
6 loader = torch.utils.data.DataLoader(
7     torch.utils.data.TensorDataset(X_all, Y_all), batch_size=32, shuffle=True)
8
9 model = ParallelLinear(3, 2)
10 loss_fn = nn.MSELoss()
11 optimizer = torch.optim.SGD(model.parameters(), lr=0.1)
12
13 for epoch in range(5):
14     model.train()
15     for X, y in loader:
16         optimizer.zero_grad()
```

```
0 16.316
1 3.096
2 0.53
3 0.145
4 0.035
```

Reference: the training loop

```
1 model = Model()                                # any nn.Module subclass
2 loss_fn = nn.MSELoss()                          # any loss
3 optimizer = torch.optim.SGD(model.parameters(), lr=0.1) # any optimizer
4
5 for epoch in range(num_epochs):
6     model.train()
7     for X, y in train_loader:
8         optimizer.zero_grad()                  # 1. clear old gradients
9         y_hat = model(X)                       # 2. forward pass
10        loss = loss_fn(y_hat, y)               # 3. score it
11        loss.backward()                        # 4. gradients
12        optimizer.step()                      # 5. update
13
14    model.eval()
15    with torch.no_grad():                      # evaluation: predictions only
16        for X, y in val_loader:
```

Making it easier on ourselves

Optional packaging on top of the interface

The loop needs three things

- Our loop needs the **model**, the **loss function**, and the **optimizer**, passed around together
- Swap the model and we usually swap the loss too. A classifier needs cross-entropy, a regressor needs MSE
- **Idea:** let the model carry its own loss and its own optimizer setup

Note

This is an *optional convenience*. Nothing in PyTorch requires it. It is what d2l does, and what our earlier demos did with `model.loss`.

Subclassing to add features

```
1 class Regressor(nn.Module):
2     def __init__(self, lr=0.1):
3         super().__init__()
4         self.lr = lr
5
6     def loss(self, y_hat, y):
7         return F.mse_loss(y_hat, y)
8
9     def configure_optimizers(self):
10        return torch.optim.SGD(self.parameters(), lr=self.lr)
11
12    def training_step(self, batch):
13        X, y = batch
14        return self.loss(self(X), y)
```

- Regressor **adds** `loss`, `configure_optimizers`, and `training_step` to `nn.Module`
- It has no `forward` yet: a subclass supplies that

A concrete model, one subclass down

```
1 class MLPRegressor(Regressor):
2     def __init__(self, num_features, num_hidden, num_outputs, lr=0.1):
3         super().__init__(lr)          # parent sets up lr
4         self.hidden = nn.Linear(num_features, num_hidden)
5         self.out = nn.Linear(num_hidden, num_outputs)
6
7     def forward(self, X):
8         return self.out(F.relu(self.hidden(X)))
```

- MLPRegressor is a Regressor is an nn.Module
- It inherits the loss and optimizer setup, and only writes what is specific to it: the layers and forward

The loop, packaged

Explicit

```
1 optimizer = SGD(model.parameters(), lr=0.1)
2 for X, y in loader:
3     optimizer.zero_grad()
4     loss = loss_fn(model(X), y)
5     loss.backward()
6     optimizer.step()
```

Packaged

```
1 optimizer = model.configure_optimizers()
2 for batch in loader:
3     optimizer.zero_grad()
4     loss = model.training_step(batch)
5     loss.backward()
6     optimizer.step()
```

- The **same five steps**. The packaged version only moves the loss and optimizer choices *into the model*

Why subclassing pays off

```
1 class Classifier(Regressor):  
2     def loss(self, y_hat, y):  
3         return F.cross_entropy(y_hat, y)
```

- One method overridden, and **the same loop** now trains a classifier
- The loop never changes. The model says what its own loss is



Tip

Reading d2l: its `Module` class is this idea. `training_step`, `loss`, and `configure_optimizers` are packaging, not requirements of the interface.

Reference: packaging with subclasses

```
1 class Regressor(nn.Module):                                # adds training conveniences
2     def __init__(self, lr=0.1):
3         super().__init__()
4         self.lr = lr
5
6     def loss(self, y_hat, y):
7         return F.mse_loss(y_hat, y)
8
9     def configure_optimizers(self):
10        return torch.optim.SGD(self.parameters(), lr=self.lr)
11
12    def training_step(self, batch):
13        X, y = batch
14        return self.loss(self(X), y)
15
16
```

Summary

The recipe

Piece	What it is	What we write
Model	an <code>nn.Module</code> subclass	<code>__init__</code> (build) and <code>forward</code> (compute)
Loss	a function of $\hat{y}$ and y	<code>nn.MSELoss()</code> , <code>F.cross_entropy</code> , ...
Optimizer	takes <code>model.parameters()</code>	<code>torch.optim.SGD(...)</code> , <code>Adam(...)</code>
Loop	five lines per batch	zero, forward, loss, backward, step
Packaging (<i>optional</i>)	subclass adds <code>loss</code> etc.	<code>training_step</code> , <code>configure_optimizers</code>

Going forward, **every model in this course is an `nn.Module` subclass** with this shape. Convolutions, recurrent layers, and attention are more blocks with the same interface.

Reference: everything on one page

```
1 import torch
2 from torch import nn
3 import torch.nn.functional as F
4
5 class Model(nn.Module):
6     def __init__(self):
7         super().__init__()          # always first
8         self.hidden = nn.Linear(3, 4) # sub-models / parameters
9         self.out = nn.Linear(4, 2)
10
11     def forward(self, X):             # inputs -> predictions
12         return self.out(F.relu(self.hidden(X)))
13
14 model = Model()
15 loss_fn = nn.MSELoss()
16 optimizer = torch.optim.SGD(model.parameters(), lr=0.1)
```